

Най-къси пътища в тегловни графи

1. Задачата за намиране на най-къси пътища — определение и варианти

Основни понятия

Работим с **ориентиран тегловен граф** $G = (V, E)$, където върху ребрата е дефинирана **тегловна функция** $w : E \rightarrow \mathbb{R}$. Под "тегло на път" разбираме сумата от теглата на ребрата му: $w(p) = \sum_{e \in E(p)} w(e)$

Важна терминологична уговорка: "най-къс път" означава "път с минимално тегло" (а не "път с най-малко ребра"). Английското "shortest path" се е наложило, въпреки че по-точно би било "най-лек път".

Дефинираме **разстоянието** $\delta(u, v)$ като теглото на най-къс път от u до v : $\delta(u, v) = \min\{w(p) \mid p \text{ е път от } u \text{ до } v\}$

Ако няма път от u до v , дефинираме $\delta(u, v) = \infty$. Ако в графа има отрицателен цикъл по някой път от u до v , тогава $\delta(u, v) = -\infty$. Затова в общия случай $\delta(u, v) \in \mathbb{R} \cup \{-\infty, +\infty\}$.

Защо не настояваме пътищата да са прости?

Когато няма отрицателни цикли, най-късите пътища така или иначе се оказват прости — повторение на връх би означавало цикъл, а изтриването му не увеличава теглото (при ≥ 0 цикли). Поради това **не дефинираме пътищата като прости**, а оставяме това да следва от свойствата на теглата. Това е важно за коректността на доказателствата.

Четири варианта на задачата

Вариант	Дадено	Търси се
Single Pair (SPShP)	G, w, s, t	$\delta(s, t)$
Single Source (SSShP)	G, w, s	$\delta(s, v)$ за всеки v
Single Destination (SDShP)	G, w, t	$\delta(v, t)$ за всеки v
All Pairs (APShP)	G, w	$\delta(u, v)$ за всяка двойка

Защо разглеждаме предимно SSShP? Защото:

- **SPShP** не е по-лесна от SSShP — в най-лошия случай, за да изчислим $\delta(s, t)$, трябва да минем през всички върхове, така че асимптотично решаваме SSShP. Няма известен алгоритъм, който решава SPShP по-бързо от SSShP.
- **SDShP** е същата като SSShP след **транспониране на графа** (обръщане на посоките на ребрата).
- **APShP** може да се реши, като пуснем SSShP алгоритъм n пъти, но за нея има специализирани алгоритми като Floyd-Warshall.

Олекотен и тежък вариант

- **Олекотен вариант:** търсим само *теглата* $\delta(s, v)$.
- **Тежък вариант:** търсим самите *пътища*, представени чрез **дърво на най-късите пътища** — масив на предшествия π , където $\pi[v]$ е родителят на v в дървото.

Дървото на най-късите пътища с корен s компактно представя всички най-къси пътища от s до останалите върхове. Има $n - 1$ ребра (при свързан граф) и спестява място — иначе би трябвало да съхраним до $n - 1$ пътища отделно.

Ключова теорема: оптимална структура

Теорема: *Най-къс път се състои от най-къси подпътища.*

Ако p е най-къс път $s \rightsquigarrow t$ и съдържа подпът q от u до v , то q е най-къс път $u \rightsquigarrow v$.

Защо? Ако имаше по-лек подпът q' от u до v , можем да заменим q с q' в p и да получим по-лек път $s \rightsquigarrow t$ — противоречие.

Това свойство е **основополагащо**: всички алгоритми за най-къси пътища стъпват върху него. То е аналогично на принципа на Белман за оптималност в динамичното програмиране.

2. Потенциални проблеми при отрицателни тегла

Защо отрицателните тегла са проблемни?

Когато допускаме отрицателни тегла, могат да се появят **отрицателни цикли** — цикли с отрицателна сума на теглата. Те създават три категорични проблема:

Проблем 1: $\delta(s, v)$ може да не е дефинирано

Ако от s има път до v , който минава през отрицателен цикъл, всяко "завъртане" през цикъла дава по-къс път. Поради това $\delta(s, v) = -\infty$ — няма минимум.

Проблем 2: Не може да се "отървем" от отрицателните тегла чрез прибавяне на константа

При **минимални покриващи дървета (МПД)** е лесно: ако теглата ни притесняват, добавяме $|x| + 1$ към всички тегла (където x е най-малкото), всички стават положителни, а МПД-то остава същото.

Този трик не работи за най-къси пътища! Прибавянето на константа c към теглото на всяко ребро променя теглото на пътя с $c \cdot |E(p)|$ — пътищата с повече ребра поскъпват повече от тези с по-малко ребра. Така най-късото може да се промени.

Пример (от учебника): Графът

```
s → a → u → v → z → t    (5 ребра с тегла 1, 1, 1, 1, 1)
s → t                      (директно ребро с тегло 5)
със страничен възел a → ... → t (с тегло -1000)
```

След прибавяне на 1001 към всички тегла, преди евтиният път през отрицателното ребро става скъп — съотношенията се променят.

Проблем 3: Защо не настояваме само за прости пътища?

Теоретично може, но **алгоритмично** е трудно. Нашите алгоритми не правят явна проверка за повторение на върхове. Освен това, задачата за **най-дълъг прост път** е NP-трудна. А смяната на знаците на теглата превръща задачата за най-къси пътища в задача за най-дълги пътища. Това подсказва, че решение чрез настояване за прости пътища би изисквало експоненциално време.

Как се справят отделните алгоритми с отрицателни тегла?

Алгоритъм	Допуска отрицателни тегла?	Допуска отрицателни цикли?
Dijkstra	Не — работи некоректно дори при едно отрицателно ребро	Не
DAG SSShP	Да — даговете нямат цикли	Не може да има (граф е даг)
Bellman–Ford	Да	Открива и сигнализира с <code>False</code>
Floyd–Warshall	Да	Открива по диагонала на матрицата

Защо отрицателните тегла все пак са полезни?

Има реални задачи, в които отрицателните тегла са естествени. **Класически пример: валутен арбитраж.** Ако имаме матрица с обменни курсове, искаме да открием поредица от обмени, която ни прави печалба: $M[i_1, i_2] \cdot M[i_2, i_3] \cdot \dots \cdot M[i_k, i_1] > 1$

Логаритмуваме и получаваме: $-\log M[i_1, i_2] - \log M[i_2, i_3] - \dots < 0$

Това е точно задачата за **намиране на отрицателен цикъл** в граф с тегла $-\log M[i, j]$. Алгоритъмът на Bellman–Ford я решава.

3. Понятието релаксация

Преди да започнем с алгоритмите, въвеждаме общата техника, която всички използват.

Идеята

За всеки връх v поддържа **горна граница** $v.d$ за $\delta(s, v)$. В началото $v.d = \infty$ (нищо не знаем). По време на работата на алгоритъма $v.d$ намалява, докато стане точно равно на $\delta(s, v)$.

Инициализация — `ISS` (Initialize-Single-Source)

```
ISS(G, s):  
1  foreach v ∈ V(G):  
2      v.d ← ∞  
3      v.π ← Nil  
4  s.d ← 0
```

Самата релаксация

```
Relax((x, y)):  
1  if y.d > x.d + w((x, y)):  
2      y.d ← x.d + w((x, y))  
3      y.π ← x
```

Какво се случва? Ако чрез x можем да стигнем до y по-евтино отколкото знаем досега, актуализираме $y.d$ и запомняме x като родител на y .

Защо се казва "релаксация"?

Малко странно име! Технически *затягаме* горната граница за $y.d$, а не я отпускаме. Терминът идва от това, че проверяваме дали ограничението $y.d \leq x.d + w((x, y))$ (което е инстанция на неравенството на триъгълника) е удовлетворено. Ако е, нямаме грижа — "релаксираме" се по отношение на него.

Свойства на релаксацията (ключови лема за всички доказателства)

Лема 45 (Неравенство на триъгълника): За всяко ребро (x, y) : $\delta(s, y) \leq \delta(s, x) + w((x, y))$

Лема 46 (Горна граница): След произволна редица от релаксации, за всеки връх v : $v.d \geq \delta(s, v)$

Освен това, ако веднъж $v.d = \delta(s, v)$, то остава такова до края.

Лема 47 (При липса на пътища): Ако няма път $s \rightsquigarrow v$, тогава $v.d = \infty$ остава завинаги.

Лема 48 (Конвергенция): Нека p е най-къс път $s \rightsquigarrow v$ и u е предпоследният връх в p . Ако в някакъв момент *преди* релаксацията на реброто (u, v) имаме $u.d = \delta(s, u)$, то *след* тази релаксация $v.d = \delta(s, v)$.

Интуиция за Лема 48: Тя казва "ако родителят е намерил истинското си разстояние и след това минем по реброто към детето, и детето намира истинското си разстояние." Това е *основополагащо* — всички доказателства за коректност съдържат в сърцевината си прилагане на Лема 48.

4. Алгоритъм на Дийкстра

Вариант на задачата

Решава **SSShP** при ограничението $w : E \rightarrow \mathbb{R}^+$ (**строго положителни тегла**).

Внимание: Алгоритъмът на Дийкстра не работи коректно дори при едно-единствено отрицателно тегло, дори ако няма отрицателни цикли.

Псевдокод (базов вариант)

```
SSShP_Dijkstra1(G, w, s):
1  ISS(G, s)
2  D ← ∅
3  while V \ D ≠ ∅:
4      избери x ∈ V \ D, такъв че x.d е минимално
5      D ← D ∪ {x}
6      foreach y ∈ adj[x]:
7          Relax((x, y))
```

Псевдокод (изтънчен вариант с приоритетна опашка)

```
SSShP_Dijkstra2(G, w, s):
1  ISS(G, s)
2  D ← ∅
3  създай празна приоритетна min-опашка Q
4  Q ← V
5  while not isEmpty(Q):
6      x ← Extract-Min(Q)
7      D ← D ∪ {x}
8      foreach y ∈ adj[x]:
9          Relax((x, y))
```

Интуиция

Дийкстра е **алчен алгоритъм**. На всяка стъпка взима връх x извън D с минимална оценка $x.d$ и го добавя в D . Идеята: щом сме сигурни, че знаем точното разстояние до x , можем да го "финализираме" и да го използваме за релаксиране на ребрата му.

Върховете се класифицират в три групи:

- **дървесни** (в D): финализирани са, знаем точно $\delta(s, v)$
- **гранични** (съседни на дървесните, но извън D): имаме оценка $v.d < \infty$
- **неизвестни** (без оценка все още): $v.d = \infty$

Картинката е аналогична на BFS, само че разширяваме границата по тегло, а не по брой ребра.

Защо изискването $w \geq 0$ е съществено?

Сърцевината на алчния избор: когато извадим x от опашката, **гарантираме**, че никакъв друг път не може да направи x по-евтин. Това разчита на факта, че всеки път да премине през x да направим обикаляне през друг връх u с $u.d > x.d$ и да се върнем... но след това трябва да добавим тегло ≥ 0 , така че няма как да стигнем до x по-евтино.

При отрицателно тегло това разсъждение се проваля.

Доказателство за коректност (Теорема 101)

Твърдение: За всеки връх v , в момента, в който v влиза в D , е изпълнено $v.d = \delta(s, v)$.

Доказателство (с допускане на противното):

Допускаме, че съществува връх v , който влиза в D с $v.d \neq \delta(s, v)$. Нека v е *първият* такъв връх. Знаем, че $v.d \geq \delta(s, v)$ (Лема 46), значи $v.d > \delta(s, v)$.

Очевидно $v \neq s$, защото s влиза първи с $s.d = 0 = \delta(s, s)$.

Разглеждаме момента t точно преди v да влезе в D . Нека p е най-къс път $s \rightsquigarrow v$ (такъв съществува заради положителните тегла). Тръгваме по p от s и намираме **първия** връх u , който още не е в D :

$s \rightarrow \dots \rightarrow a \rightarrow u \rightarrow \dots \rightarrow v$
в D не е в D , не е в D

(Тук a е предшественикът на u в p и е в D . Възможно е $a = s$.)

Ключово твърдение: $u.d = \delta(s, u)$ в момент t .

Защо? Когато a е влязъл в D (на по-ранен момент $t' < t$), $a.d$ е било $\delta(s, a)$ (защото v е първият грешен връх, така че a не греши). Веднага след това реброто (a, u) е релаксирано. По **Лема 48 (Конвергенция)** $u.d$ става $\delta(s, u)$ след тази релаксация и остава такова до момент t .

Сега комбинираме:

1. $u.d = \delta(s, u)$ в момент t (току-що показахме)
2. $\delta(s, u) \leq \delta(s, v)$ (положителни тегла + оптимална подструктура; u е по пътя p преди v)
3. $\delta(s, v) < v.d$ (нашето допускане)

Така че: $u.d < v.d$.

Но Дийкстра избра v , а не u , при положение че и двата са в $V \setminus D$. Алгоритъмът избира връх с **минимална** $.d$ стойност, значи $v.d \leq u.d$.

Получихме верига: $v.d \leq u.d = \delta(s, u) \leq \delta(s, v) < v.d$

Това е невъзможно. ■

Изследване на сложността по време

Сложността зависи **силно** от това как реализираме операцията "избери връх с минимална $.d$ ".

Базов вариант (последователно търсене)

- While-цикълът се изпълнява n пъти.
- Всяко изпълнение търси минимум в $V \setminus D$ — това е $\Theta(n)$.
- Релаксацията общо за целия алгоритъм е $\Theta(m)$ (всяко ребро се релаксира веднъж).
- Общо: $\Theta(n^2 + m) = \Theta(n^2)$.

Кога е добър? За **плътни графи**, където $m \approx n^2$. Тогава $\Theta(n^2)$ е оптимално, защото и без друго трябва да прочетем входа.

Изтънчен вариант с двоична пирамида

- Извличане на минимум: $\Theta(\log n)$, прави се n пъти $\rightarrow \Theta(n \log n)$

- Релаксация: при намаляване на ключ — $\Theta(\log n)$, прави се най-много m пъти $\rightarrow \Theta(m \log n)$
- Общо: $\Theta((n + m) \log n)$, за свързан граф това е $\Theta(m \log n)$.

Кога е добър? За **разредени графи**, където $m = O(n)$ или $m = O(n \log n)$.

Изтънчен вариант с пирамида на Fibonacci

- Извличане на минимум: $O(\log n)$ амортизирано, n пъти $\rightarrow O(n \log n)$
- Decrease-Key: $O(1)$ амортизирано, m пъти $\rightarrow O(m)$
- Общо: $O(m + n \log n)$.

Това е асимптотично най-доброто, известно за общия случай.

Сравнение

Реализация	Сложност
Масив + линейно търсене	$\Theta(n^2)$
Двоична пирамида	$\Theta((n + m) \log n)$
Пирамида на Fibonacci	$O(m + n \log n)$

Свойство: върховете влизат в дървото в ненамаляващ ред на разстоянията

Теорема 102: При всяко достигане на while-цикъла, за всеки $u \in D$ и $v \in V \setminus D$ имаме $u.d \leq v.d$.

Това означава, че Дийкстра обхожда върховете "по сферични вълни" от s — първо най-близкия, после следващия по близост, и така нататък. Аналог на BFS, но с тегла.

5. Алгоритъм за най-къси пътища в тегловен ДАГ

Вариант на задачата

Решава **SSShP** в **ориентиран ацикличен граф** (ДАГ). Не изисква положителни тегла — допускат се **отрицателни**, защото в ДАГ няма цикли и съответно няма отрицателни цикли.

Псевдокод

```
DAG_SSShP(G, w, s):  
1  Toposort(G)  
2  ISS(G, s)  
3  foreach x ∈ V(G) в топологичен ред:  
4      foreach y ∈ adj[x]:  
5          Relax((x, y))
```

Интуиция

В ДАГ ребрата винаги вървят "напред" в топологичния ред. Затова, ако обработваме върховете в този ред и релаксираме всички изходящи ребра при всеки връх, когато стигнем до връх v , **всички** възможни пътища $s \rightsquigarrow v$ вече са разгледани — защото всеки предпоследен връх в такъв път е вляво от v , тоест бил е обработен преди.

Това е особено елегантно: **нямаме нужда** да избираме върхове с минимална d , нямаме нужда от приоритетна опашка. Топологичният ред "ни казва" в каква последователност да работим.

Доказателство за коректност (Теорема 103)

Първо доказателство (по индукция върху позицията в топологичния ред):

- За върхове v вляво от s :** Няма път $s \rightsquigarrow v$ (защото ребрата вървят напред). Значи $\delta(s, v) = \infty$ и $v.d = \infty$ цял живот (Лема 47). ✓
- За върха s :** $s.d = 0 = \delta(s, s)$ след `ISS`, остава такова до края (Лема 46). ✓
- За произволен връх v вдясно от s :** Нека U е множеството от предпоследни върхове на пътищата $s \rightsquigarrow v$. Всички такива върхове са в топологичния ред между s и v . От **алгоритмични съображения** (релаксацията на ребрата от U към v ги обхожда): $v.d = \min_{u \in U} \{u.d + w((u, v))\}$ От **граф-теоретични съображения** (оптимална подструктура): $\delta(s, v) = \min_{u \in U} \{\delta(s, u) + w((u, v))\}$ По индуктивно предположение, $u.d = \delta(s, u)$ за всеки $u \in U$ (всички са по-наляво от v и вече обработени). Заклучаваме $v.d = \delta(s, v)$. ■

Изследване на сложността

- Топологичното сортиране е $\Theta(n + m)$.
- Вложеният for-цикъл (редове 3–5) минава по всички списъци на съседство, тоест работи $\Theta(n + m)$.

- Общо: $\Theta(n + m)$.

Това е **оптимално асимптотично** — не може да се направи по-бързо, защото поне трябва да прочетем входа.

Относителни предимства пред Dijkstra върху ДАГ

Свойство	Dijkstra (даже с Fibonacci heap)	DAG SSShP
Сложност	$O(m + n \log n)$	$\Theta(n + m)$
Работи с отрицателни тегла	Не	Да
Намира и най-дълги пътища	Не	Да (просто обръщаме знака на неравенството в Relax)

Защо DAG SSShP може да намира и най-дълги пътища, а Dijkstra — не?

В ДАГ е валидно, че **най-дълъг път се състои от най-дълги подпътища**. Заменяме подпът с по-дълъг — няма опасност да получим непрост път (защото ДАГ няма цикли). При общи графи това свойство не е валидно, защото може да получим повторение на върхове и неограничено увеличаване на дължината.

При Dijkstra смяната `min → max` не работи. Защо? Защото алчният избор разчита, че щом извадим минималния връх, никой друг не може да го "удължи" обратно надолу. При най-дълги пътища такава гаранция няма дори при положителни тегла.

6. Алгоритъм на Белман–Форд

Вариант на задачата

Решава **SSShP** с **произволни реални тегла**. Открива наличието на отрицателен цикъл, достижим от s , и сигнализира с `False`.

Псевдокод

```
Bellman_Ford(G, w, s):
1  ISS(G, s)
2  for i ← 1 to n - 1:
3      foreach (x, y) ∈ E(G):
4          Relax((x, y))
5  foreach (x, y) ∈ E(G):
6      if y.d > x.d + w((x, y)):
7          return False
8  return True
```

Интуиция

Идеята е проста и красива: $n - 1$ пъти релаксираме всички ребра в произволен ред.

Защо $n - 1$? Защото:

- Дървото на най-късите пътища има най-много $n - 1$ ребра.
- Най-дълъг прост път има най-много $n - 1$ ребра.
- На всяка итерация на цикъла "разпространяваме" окончателните стойности с поне още едно ниво в дървото на най-късите пътища.

При най-добрия случай всички окончателни стойности могат да се изчислят още на първата итерация (ако случайно релаксираме ребрата в добър ред). Но в най-лошия случай ни трябва точно $n - 1$ итерации.

Доказателство за коректност (случай: няма отрицателни цикли)

Инвариант (Инвариант 31 за for-цикъла на редове 2–4):

При всяко достигане на ред 2, за всеки връх v от T_{i-1} е изпълнено $v.d = \delta(s, v)$.

Тук T е дървото на най-късите пътища с корен s (което съществува, защото няма отрицателни цикли), а T_k е поддървото му, индуцирано от върховете на дълбочина $\leq k$.

База ($i = 1$): $T_0 = \{s\}$. $s.d = 0 = \delta(s, s)$. ✓

Поддръжка: Допускаме, че при достигане на ред 2 в стъпка i , инвариантът е в сила за T_{i-1} . Разглеждаме произволен връх t от ниво i . Той има родител v в ниво $i - 1$, а реброто (v, t) е в дървото на най-късите пътища.

На итерация i цикълът `foreach` (редове 3–4) релаксира всички ребра, включително (v, t) . По индуктивно предположение $v.d = \delta(s, v)$. По **Лема 48 (Конвергенция)**, след релаксацията на (v, t) имаме $t.d = \delta(s, t)$, което остава до края. ✓

Терминация: След последното изпълнение на `for`-цикъла, всички върхове на дървото имат окончателни стойности. ■

Следствие: В тази ситуация условието на ред 6 винаги е лъжа, така че алгоритъмът връща `True`.

Установяване на отрицателен цикъл (Теорема 105)

Лема 51: Ако в G има отрицателен цикъл, тогава след `for`-цикъла на редове 2–4 съществува поне едно ребро (x, y) , за което $y.d > x.d + w((x, y))$.

Доказателство: Допускаме противното — за всяко ребро (x, y) имаме $y.d \leq x.d + w((x, y))$. Разглеждаме произволен отрицателен цикъл $c = v_1, e_1, v_2, e_2, \dots, v_k, e_k, v_1$.

Записваме неравенствата: $v_2.d \leq v_1.d + w(e_1)$ $v_3.d \leq v_2.d + w(e_2)$: $v_1.d \leq v_k.d + w(e_k)$

Сумираме всички k неравенства. Лявата страна е $v_1.d + v_2.d + \dots + v_k.d$, дясната страна е същата сума плюс $w(e_1) + w(e_2) + \dots + w(e_k)$.

Съкращавайки получаваме: $0 \leq w(e_1) + w(e_2) + \dots + w(e_k)$

Но c е отрицателен цикъл, значи сумата му е < 0 . Противоречие! ■

Интуиция: Ако всички ограничения " $y.d \leq x.d + w$ " са изпълнени за всяко ребро на цикъла, сумираме тях и d -стойностите се съкращават; остава неравенство върху сумата на теглата, което не може да се удовлетвори при отрицателен цикъл.

Изследване на сложността по време

- `For`-цикълът (редове 2–4) се изпълнява $n - 1$ пъти.
- Във всяка итерация се обработват всички m ребра.
- Общо: $\Theta(n \cdot m)$.

Това е **кубично в най-лошия случай** (при $m \approx n^2$). Belman-Ford е драстично по-бавен от Dijkstra и DAG SSShP — цената на това, че се справя с отрицателни тегла.

7. Алгоритъм на Флойд–Уоршал

Вариант на задачата

Решава **All Pairs Shortest Paths (APShP)** — намира $\delta(u, v)$ за **всяка двойка** върхове. Допуска отрицателни тегла, но не отрицателни цикли (открива ги и сигнализира).

Основната идея — динамично програмиране

Алгоритъмът е конструиран по схемата **Динамично програмиране**. Идеята: номерираме върховете $\{1, 2, \dots, n\}$ и дефинираме:

$d_{ij}^{(k)}$ = тегло на най-къс път от i до j , чиито вътрешни върхове са от $\{1, 2, \dots, k\}$

"Вътрешни върхове" означава върхове в пътя, без началния и крайния. Например, ако път е $i \rightarrow 3 \rightarrow 7 \rightarrow j$, вътрешните върхове са $\{3, 7\}$.

Рекурентното отношение

При $k = 0$: пътищата нямат вътрешни върхове, тоест те са директни ребра: $d_{ij}^{(0)} = w(i, j)$ (или ∞ , ако няма ребро)

За $k \geq 1$ имаме фундаменталния избор:

$$d_{ij}^{(k)} = \min \left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right)$$

Защо това е вярно?

Разглеждаме най-къс път от i до j с вътрешни върхове от $\{1, \dots, k\}$. Има два случая:

1. **k не е вътрешен връх в пътя.** Тогава пътят използва само върхове от $\{1, \dots, k-1\}$ като вътрешни, и теглото му е $d_{ij}^{(k-1)}$.
2. **k е вътрешен връх в пътя.** Тогава пътят се разделя на две части: $i \rightarrow \dots \rightarrow k$ и $k \rightarrow \dots \rightarrow j$. И в двете части k не се повтаря като вътрешен (иначе би имало цикъл, който не подобрява пътя при липса на отрицателни цикли). Така че вътрешните върхове на двете части са от $\{1, \dots, k-1\}$, и общото тегло е $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.

Взимаме минимума от двата случая.

Окончателният отговор за разстоянието е: $\delta(i, j) = d_{ij}^{(n)}$

защото при $k = n$ са допустими всички възможни вътрешни върхове.

Псевдокод

```
Floyd_Warshall(W: матрица n×n с тегла):  
1  D(0) ← W  
2  for k ← 1 to n:  
3    for i ← 1 to n:  
4      for j ← 1 to n:  
5        D(k)[i, j] ← min( D(k-1)[i, j], D(k-1)[i, k] + D(k-1)[k, j] )  
6  return D(n)
```

Имплементация с една матрица

Може да забележим, че можем да работим с **една единствена матрица** D "на място".

Защо? Защото при актуализиране на $D[i, j]$ ползваме $D[i, k]$ и $D[k, j]$. Оказва се, че:

- $D[i, k]$ в момента k : $D[i, k]^{(k)} = \min(D[i, k]^{(k-1)}, D[i, k]^{(k-1)} + D[k, k]^{(k-1)})$. Тъй като $D[k, k]^{(k-1)} \geq 0$ (при липса на отрицателни цикли), имаме $D[i, k]^{(k)} = D[i, k]^{(k-1)}$.
- Аналогично, $D[k, j]^{(k)} = D[k, j]^{(k-1)}$.

Тоест ред k и стълб k не се променят между фази $k - 1$ и k . Така че можем да актуализираме матрицата на място, без да копираме.

```
Floyd_Warshall_inplace(D):  
1  for k ← 1 to n:  
2    for i ← 1 to n:  
3      for j ← 1 to n:  
4        if D[i, k] + D[k, j] < D[i, j]:  
5          D[i, j] ← D[i, k] + D[k, j]  
6  return D
```

Доказателство за коректност

Твърдение: След край на алгоритъма, $D[i, j] = \delta(i, j)$ за всички i, j .

Доказателство по индукция върху k :

Инвариант: В края на итерация k от външния цикъл, за всички i, j : $D[i, j] = d_{ij}^{(k)}$.

База ($k = 0$): преди първата итерация, $D = W$, което е именно $d_{ij}^{(0)}$. ✓

Поддръжка: Допускаме, че в края на итерация $k - 1$ матрицата съдържа $d_{ij}^{(k-1)}$ за всички i, j . При итерация k , за всяка двойка (i, j) изчисляваме: $D[i, j] \leftarrow \min(D[i, j], D[i, k] + D[k, j]) = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) = d_{ij}^{(k)}$

(Тук се позоваваме на наблюдението по-горе, че $D[i, k]$ и $D[k, j]$ не се променят в текущата итерация.) ✓

Терминация: При $k = n$, $D[i, j] = d_{ij}^{(n)} = \delta(i, j)$. ■

Откриване на отрицателни цикли

Ако след края на алгоритъма за някое i имаме $D[i, i] < 0$, то i е на отрицателен цикъл.

Интуиция: $D[i, i]$ е теглото на най-евтиния "път" от i до самия себе си с поне едно ребро — тоест на най-евтиния цикъл, минаващ през i . При липса на отрицателни цикли това е ≥ 0 (всъщност е 0 за пътя с дължина 0).

Изследване на сложността по време

- Три вложени цикъла, всеки n итерации.
- Тялото на най-вътрешния цикъл прави $\Theta(1)$ работа.
- Общо: $\Theta(n^3)$.

Кубично, независимо от m . Това е особеност на Floyd-Warshall: не се възползва от разредеността на графа. Работи и при $m = 0$ кубично!

Сложност по памет

В **наивна** реализация съхраняваме $n + 1$ матрици $D^{(0)}, D^{(1)}, \dots, D^{(n)}$, всяка $n \times n$. Това е $\Theta(n^3)$ памет.

В **подобрена** реализация (на място) използваме само **една матрица D** , която обновяваме итеративно. Памет: $\Theta(n^2)$.

За реконструкция на пътищата (тежък вариант) поддържа допълнителна матрица Π , $n \times n$, която съхранява предшествениците. Памет остава $\Theta(n^2)$.

Сравнение с други алгоритми за APShP

Подход	Сложност по време
Dijkstra от всеки връх (с пирамида на Fibonacci)	$O(n(m + n \log n)) = O(nm + n^2 \log n)$
Bellman-Ford от всеки връх	$O(n^2 m)$
Johnson (използва Bellman-Ford + Dijkstra)	$O(n^2 \log n + nm)$
Floyd-Warshall	$\Theta(n^3)$

За **плътни графи** ($m \approx n^2$): Floyd-Warshall (n^3) се справя добре, конкурира се с Dijkstra от всеки връх ($nm = n^3$).

За **разредени графи** с положителни тегла: Dijkstra от всеки връх е по-добър ($n^2 \log n + nm$).

Силни страни на Floyd-Warshall:

- Изключително проста реализация — само три вложени цикъла.
- Работи с отрицателни тегла (Dijkstra не работи).
- Открива отрицателни цикли естествено.
- Сложността не зависи от m — добро за плътни графи.
- Често по-бърз на практика заради малките константи и добрия cache locality.

Обобщение: Кога кой алгоритъм да използваме?

```
Имам тегловен граф G и трябва да намеря най-къси пътища.
|
├─ APShP (всички двойки)?
|   ├─ Плътен граф или прости имплементации → Floyd-Warshall:  $\Theta(n^3)$ 
|   ├─ Разредени графи с произволни тегла → Johnson:  $O(n^2 \log n + nm)$ 
|   └─ Положителни тегла → Dijkstra от всеки връх
|
└─ SSShP (от един връх)?
    ├─ G е ДАГ → DAG SSShP:  $\Theta(n + m)$ 
    ├─ Има отрицателни тегла → Bellman-Ford:  $\Theta(nm)$ 
    └─ Положителни тегла → Dijkstra
        ├─ Плътен граф → базов вариант:  $\Theta(n^2)$ 
        └─ Разреден граф → с пирамида:  $O(m \log n)$  или  $O(m + n \log n)$ 
```

Ключови интуиции, които да си запомним

1. **Релаксацията** е централна техника — всички алгоритми всъщност просто релаксират ребра в различен ред.
2. **Лема 48 (Конвергенция)** е сърцето на всички доказателства: щом родителят знае истинското си разстояние и след това минем по реброто към детето, детето научава своето.
3. **Dijkstra** е алчен: разчита, че положителните тегла никога не "ни връщат назад" в евтиността.
4. **DAG SSShP** е "Dijkstra без приоритетна опашка" — топологичният ред върши работата вместо нас.
5. **Bellman-Ford** е "грубата сила" — повтаряме релаксации, докато конвергират; работи винаги, но е бавен.
6. **Floyd-Warshall** е "блестящо динамично програмиране" — добавяме един по един възможните вътрешни върхове.
7. **Отрицателните цикли** са истинският враг — те правят задачата неопределена. Поне един от алгоритмите (Bellman-Ford, Floyd-Warshall) може да ги открие.